购物（shopping）

本题正解所涉及的知识点主要包括贪心动态规划与背包动态规划，其中“贪心”属于“基础算法”，大纲标注学习难度系数为3；“背包”属于“线性DP-背包”形态的规划，大纲标注学习难度系数为5。本题所涉及的知识点均在NOI大纲所规定的CSP-S难度系数范围内。

测试点1~2：$n \leq 15, m \leq 200$

思路

由于商品数量很少（不超过15），我们可以枚举所有可能的商品子集，然后检查这个子集是否存在一个合法的购买顺序。

具体地，对于一个子集，我们将其中的商品按价格从大到小排序（因为先买贵的更容易满足条件），然后依次模拟购买过程：维护当前剩余钱数 rem，对于每一件商品，如果它的价格 a 满足 $2 * a > rem$，则说明这件商品不能购买，该子集不合法；否则就买下它，$rem-=a$，累加价值。如果整个子集都能按这个顺序枚举所有 2^n 个子集，每个子集排序后检查，时间复杂度可以接受。

复杂度

- 时间复杂度：$O(2^n \cdot n \log n)$，其中 $n \leq 15$，完全可行。
- 空间复杂度：$O(n)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m, ans = 0;
struct node { int a, b; } g[20], t[20];
int s[20];  // s[i] 表示当前已选商品的下标
bool cmp(node x, node y) { return x.a > y.a; }
void check(int c) {
    for (int i = 0; i < c; ++i) t[i] = g[s[i]];
    sort(t, t + c, cmp);
    int rem = m, val = 0;
    for (int i = 0; i < c; ++i) {
        if (2 * t[i].a > rem) return;   // 不合法直接返回
        rem -= t[i].a;
        val += t[i].b;
    }
    if (val > ans) ans = val;
}
void dfs(int i, int c) {
    if (i == n) { check(c); return; }
    dfs(i + 1, c);              // 不选第 i 件商品
    s[c] = i;
    dfs(i + 1, c + 1);          // 选第 i 件商品
}
int main() {
    cin >> n >> m;
    for (int i = 0; i < n; ++i) cin >> g[i].a >> g[i].b;
    dfs(0, 0);
    cout << ans << '\n';
```

```
    return 0;
}
```

测试点3~6：所有 a_i 相等

思路

设所有商品的价格均为 a。如果我们要购买 k 件商品，总花费为 $k \cdot a$。购买过程中，第 j 次购买时已经花费了 $(j-1)a$，剩余钱数为 $m-(j-1)a$，根据规则需要满足：

$$a \le \frac{m-(j-1)a}{2} \tag{1}$$

即：

$$2a \le m-(j-1)a \implies (j+1)a \le m \tag{2}$$

因此，最多能够购买的商品件数 $k_{\max}$ 必须满足 $(k_{\max}+1)a \le m$，也就是 $k_{\max} = \lfloor m/a \rfloor - 1$。如果 $k_{\max} \le 0$，则一件都买不了，答案为 0。否则，为了让总价值最大，我们显然应该从所有商品中挑选价值最大的 $k_{\max}$ 件（如果不足则全取）求和即可。

复杂度

- 时间复杂度：$O(n \log n)$，主要是排序。
- 空间复杂度：$O(n)$

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> b(n);
    int a = -1;
    for (int i = 0; i < n; ++i) {
        cin >> a >> b[i];
    }
    sort(b.rbegin(), b.rend());//从大到小排
    int k = m / a - 1;  // 最多能买的件数
    if (k <= 0) {
        cout << 0 << '\n';
        return 0;
    }
    int ans = 0;
    for (int i = 0; i < min(k, n); ++i) {
        ans += b[i];
    }
    cout << ans << '\n';
    return 0;
}
```

测试点7~9：所有 b_i 相等

思路

所有商品价值相同，因此目标转化为最大化购买商品的数量。由于购买顺序可以自由决定，最优策略是按价格从大到小依次尝试购买。下面证明“价格从大到小购买”是最优的。

假设当前剩余钱数为R，有两件商品价格分别为p和q，且$p > q$。我们比较两种购买顺序的可行性。

先买便宜的q，再买贵的p：
买完q需要$p \le R/2$，即$R \ge 2p$；买q时剩余为$R-q$，需要$p \le (R-q)/2$，即$R-q \ge 2p$。合并得：
$R \ge p + 2q$

先买贵的p，再买便宜的q：
买p需要$q \le R/2$，即$R \ge 2q$；买p时剩余$R-q$，需要$p \le (R-q)/2$，即$R-q \ge 2p$。合并得：
$R \ge \max(2q,\ q+2p)$

比较两个门槛：因为$q > p > 0$，有$p+2q > 2p$，且$q+2p > q+2p$（因为$q-p>0$）。所以：
$p+2q > \max(2q,\ q+2p)$

这意味着，如果“先买贵再买便宜”可行，那么“先买便宜再买贵”一定也可行；反过来不一定。因此，在购买顺序中，如果存在相邻的逆序对（便宜在前、贵在后），交换它们之后，仍然可以购买，且不会改变已买数量。最优策略是将所有商品按价格从大到小排序，然后依次尝试购买。

设$dp[j]$表示已花费j元时能购买的最大件数。初始$dp[0]=0$，其余为负无穷。对于价格为a的商品，若当前已花费j元，要购买它必须满足：
$j + 2a \le m$
则可以进行转移：
$dp[j+a] = \max(dp[j+a],\ dp[j]+b)$
最终答案为$\max_{0 \le j \le m} dp[j]$。

复杂度

- 时间复杂度：$O(nm)$，其中 $n \le 200$，$m \le 20000$，运算量约 4×10^6。
- 空间复杂度：$O(m)$

测试点10~12：$a_i \le 100$

思路

格范围很小，意味着不同的价格种类数至多为100。我们可以将所有商品按价格分组，同一价格组内按价值从大到小排序，按价格从大到小依次处理每个价格组，这样可以保证与最优购买顺序一致（先贵后便宜）。

设$dp[j]$表示已花费j元时能获得的最大价值，初始$dp[0]=0$，其余为负无穷。对于价格为a的组，组内商品按价值排序为$b_1 \ge b_2 \ge \cdots \ge b_k$，并设前缀和$S[c] = b_1 + b_2 + \cdots + b_c$。

从已花费j元出发，若决定购买该组中价值最大的c件商品，则在购买第c件时已花费$j + (c-1)a$元，需要满足：
$j + (c-1)a + 2a \le m$
化简得：
$j + (c+1)a \le m$

在该条件成立时，可以转移到状态$j + ca$，价值增加$S[c]$：
$dp[j+ca] = \max(dp[j+ca],\ dp[j] + S[c])$

按价格从大到小处理每个组，最终答案为$\max_{0\le j\le m} dp[j]$。

复杂度

- 时间复杂度：$O(nm)$。
- 空间复杂度：$O(m)$。

测试点13~16：$b_i = a_i$

思路

当价值等于价格时，总价值等于总花费，因此只需最大化总花费。设$dp[j]$为布尔数组，表示能否恰好花掉j元。初始$dp[0] = \text{true}$，其余为false。

将所有商品按价格降序排列。对于价格为a的商品，若当前已花费j元，要购买它必须满足：
$j + 2a \le m$
在满足该条件时，若$dp[j]$为真，则令$dp[j + a]$为真。最后从m向下找第一个为真的j，这个j就是最大总花费，也就是最大总价值。

复杂度

- 时间复杂度：$O(nm)$，使用 `bitset` 可以进一步优化，但一维布尔数组已经足够。
- 空间复杂度：$O(m)$。

测试点17~20：无特殊条件

思路

这是一个经典的01背包问题，但购买有一个特殊的购买条件：必须按价格从大到小购买才合法。

关键观察：如果存在一个合法的购买顺序，则一定存在一个按价格从大到小购买的合法顺序。简要说明：假设当前剩余钱数为 R，有两件商品价格 $p > q$。先买 p 再买 q 的条件为 $p \le R/2$，$q \le (R - p)/2$；如果反过来先买 q，那么条件为 $q \le R/2$，$p \le (R - q)/2$。因为 $p > q$，先买贵的更容易满足条件。我们可以通过交换相邻逆序对，把任意合法顺序调整为从大到小。（同）

于是，我们将所有商品按价格从大到小排序，然后进行01背包。

设 $dp[j]$ 表示已花费总价为 j 时能获得的最大价值。初始 $dp[0] = 0$，其余为负无穷。

对于每个商品 (a, b)，倒序遍历 j：如果 $j + 2a \le m$ 且 $dp[j]$ 有效，则更新 $dp[j + a] = \max(dp[j + a], dp[j] + b)$。

最终答案是 $\max_{0\le j\le m} dp[j]$。

复杂度

- 时间复杂度：$O(nm)$，$n \le 200$，$m \le 20000$，约 4×10^6 次操作。
- 空间复杂度：$O(m)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
 const int inf=-1000000000;
struct node{int a,b;}g[205];
int dp[20005];
bool cmp(node x,node y){return x.a>y.a;}
```

```cpp
int main(){
    int n,m;cin>>n>>m;
    for(int i=0;i<n;++i)
        cin>>g[i].a>>g[i].b;
    sort(g,g+n,cmp);
    for(int j=0;j<=m;++j)
        dp[j]=inf;
    dp[0]=0;
    for(int i=0;i<n;++i){
        int a=g[i].a,b=g[i].b;
        for(int j=m-2*a;j>=0;--j){
            if(dp[j]!=inf)
                dp[j+a]=max(dp[j+a],dp[j]+b);
        }
    }
    int ans=0;
    for(int j=0;j<=m;++j)if(dp[j]>ans)ans=dp[j];
    cout<<ans<<endl;
    return 0;
}

```

星际航行（travel）

本题及正解部分的知识点主要包括 单源最短路（Dijkstra）、反图、位运算以及二进制分组优化，其中“单源最短路”属于“图论算法”，大纲标注学习难度系数为6；“位运算”属于“基本运算”，大纲标注学习难度系数为2。本题所涉及的知识点均在大纲所规定的CSP-S难度系数范围内。

测试点1~2：$n \leq 100, m \leq 200, k \leq n$

思路

数据范围很小，可以直接求出任意两点间的最短路。由于边权非负，使用 Floyd-Warshall 算法最为方便。

设$dis[i][j]$表示从节点i到节点j的最短路径长。初始时，对于每条边$u \to v$权值为w，令$dis[u][v] = \min(dis[u][v], w)$，且$dis[i][i] = 0$。枚举中转点$k$，若从$i$到$k$再到$j$的路径更短，则更新$dis[i][j]$。

处理完 Floyd 后，枚举每个关键点p_i，遍历其他关键点p_j $(j \neq i)$，统计$dis[p_i][p_j]$的最小值，若$k = 1$，不存在其他关键点，直接输出 -1。

复杂度

时间复杂度$O(n^3 + k^2)$，空间复杂度$O(n^2)$。

```cpp
#include <bits/stdc++.h>
using namespace std;

```

```cpp
typedef long long ll;
const ll INF = 4e18;
int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m, k;
    cin >> n >> m >> k;
    vector<vector<ll>> dist(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; ++i) dist[i][i] = 0;
    for (int i = 0; i < m; ++i) {
        int u, v; ll w; cin >> u >> v >> w;
        dist[u][v] = min(dist[u][v], w); // 重边取小的
    }
    vector<int> keys(k);
    for (int i = 0; i < k; ++i) cin >> keys[i];
    if (k == 1) { cout << -1 << "\n"; return 0; }
    for (int kk = 1; kk <= n; ++kk)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (dist[i][kk] != INF && dist[kk][j] != INF)
                    dist[i][j] = min(dist[i][j], dist[i][kk] + dist[kk][j]);

    for (int i = 0; i < k; ++i) {
        ll mn = INF;
        for (int j = 0; j < k; ++j) {
            if (i == j) continue;
            mn = min(mn, dist[keys[i]][keys[j]]);
        }
        cout << mn << "\n";
    }
    return 0;
}
```

测试点3~6：$n \le 1000, m \le 2000, k \le 100$

思路

k最大为 100，可以对每个关键点跑一次单源最短路。由于$n \le 1000$，使用$O(n^2)$的朴素 Dijkstra 算法是比较划算的，且$100 \times 1000^2 = 10^8$，在 1 秒内可以轻松跑完。
朴素 Dijkstra 的过程：维护一个$dist$数组表示起点到各点的最短距离，初始化为无穷大，起点为 0。每次从访问过的节点中选出$dist$最小的节点u，标记已访问，然后遍历所有节点v，如果$dist[u] + w(u, v) < dist[v]$，则更新。

复杂度

时间复杂度$O(k \cdot n^2)$，空间复杂度$O(n^2)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = (ll)4e18;
ll adj[1005][1005], dist[1005];
bool vis[1005];
```

```cpp
int key_[105];
int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    int n, m, k;
    cin >> n >> m >> k;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j) adj[i][j] = (i == j ? 0 : INF);
    for (int i = 0; i < m; ++i) {
        int u, v; ll w; cin >> u >> v >> w;
        adj[u][v] = min(adj[u][v], w);
    }
    for (int i = 0; i < k; ++i) cin >> key_[i];
    if (k == 1) { cout << -1 << '\n'; return 0; }
    for (int i = 0; i < k; ++i) {
        for (int j = 1; j <= n; ++j) { dist[j] = INF; vis[j] = false; }
        dist[key_[i]] = 0;
        for (int it = 1; it <= n; ++it) {
            int u = -1; ll best = INF;
            for (int v = 1; v <= n; ++v)
                if (!vis[v] && dist[v] < best) { best = dist[v]; u = v; }
            if (u == -1) break;                          // 剩下的点都到不了
            vis[u] = true;
            for (int v = 1; v <= n; ++v)
                if (!vis[v] && adj[u][v] != INF && dist[u] + adj[u][v] <
dist[v])
                    dist[v] = dist[u] + adj[u][v];
        }
        ll mn = INF;
        for (int j = 0; j < k; ++j)
            if (j != i) mn = min(mn, dist[key_[j]]);
        cout << (mn >= INF / 2 ? -1 : mn) << '\n';
    }
    return 0;
}
```

测试点7~8：$n \le 10000, m \le 20000, k \le 100$

思路

n达到了 10000，邻接矩阵会超出空间限制（$10000^2 \times 8$ 字节 > 800 MB），必须使用邻接表存储稀疏图。同时，朴素 Dijkstra 的$O(n^2)$在 10000 级别下会达到10^8次，k总计10^{10}，会超时，因此需要使用优先队列（堆）优化的 Dijkstra 算法。

堆优化 Dijkstra 的过程：将起点距离设为 0 加入优先队列，每次从队首取出距离最小的点，如果该距离大于$dist$数组中的值则跳过（懒惰删除）。遍历其邻接边，如果可以通过该点缩短到邻居的距离，则更新并加入队列。时间复杂度降为$O(m \log n)$。

复杂度

时间复杂度$O(k \cdot m \log n)$，空间复杂度$O(n + m)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
typedef long long ll;
const ll INF = (ll)4e18;
const int MAXN = 10005, MAXM = 20005;
int head[MAXN], to[MAXM], nxt[MAXM], ecnt;
ll wt[MAXM];
int key_[105];
ll dist[MAXN];
void add(int u, int v, ll w) {
    to[++ecnt] = v; wt[ecnt] = w; nxt[ecnt] = head[u]; head[u] = ecnt;
}
int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    int n, m, k;
    cin >> n >> m >> k;
    for (int i = 0; i < m; ++i) { int u, v; ll w; cin >> u >> v >> w; add(u, v, w); }
    for (int i = 0; i < k; ++i) cin >> key_[i];
    if (k == 1) { cout << -1 << '\n'; return 0; }
    typedef pair<ll, int> P;
    for (int i = 0; i < k; ++i) {
        for (int j = 1; j <= n; ++j) dist[j] = INF;
        priority_queue<P, vector<P>, greater<P>> pq;
        dist[key_[i]] = 0;
        pq.push(P(0, key_[i]));
        while (!pq.empty()) {
            P cur = pq.top(); pq.pop();
            ll d = cur.first; int u = cur.second;
            if (d != dist[u]) continue;                    // 过期的堆顶，丢掉
            for (int e = head[u]; e; e = nxt[e]) {
                int v = to[e];
                if (d + wt[e] < dist[v]) { dist[v] = d + wt[e]; pq.push(P(dist[v], v)); }
            }
        }
        ll mn = INF;
        for (int j = 0; j < k; ++j)
            if (j != i) mn = min(mn, dist[key_[j]]);
        cout << (mn >= INF / 2 ? -1 : mn) << '\n';
    }
    return 0;
}
```

测试点9~12：$n \le 100000, m \le 200000, k \le n$，关键点的下标二进制中1的个数不超过2

思路

$n \le 100000$，这一档的限制看着吓人，其实是在送分------限制的是**下标的二进制形态**，而这个限制能把 `k` 卡死在一个很小的数上。

$100000 < 2^{17} = 65536$，$2^{17} = 131072$。因此，下标最多有 17 个二进制位，二进制中 1 的个数不超过 1 的有 17 个；恰好有 2 个的有$\binom{17}{2} = 136$个。

总共有：$17+136=153$个，满足$k \le n$，即$k \le 153$。这实际上意味着这个测试点中k非常小，我们直接运行测试点7-8的k次堆优化 Dijkstra 即可。

复杂度

时间复杂度$O(k \cdot m \log n)$，其中$k \le 153$，空间复杂度$O(n+m)$。
代码复用测试点7-8的代码即可。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int MAXN = 10005;
const int MAXM = 20005;
const ll INF = 4e18;

// 链式前向星
int head[MAXN], cnt = 0;
struct Edge { int to, nxt; ll w; } e[MAXM];
void add(int u, int v, ll w) {
    e[++cnt] = {v, head[u], w};
    head[u] = cnt;
}

// 优先队列节点重载
struct Node {
    ll d; int u;
    bool operator<(const Node& o) const { return d > o.d; }
};

ll dist[MAXN];
int keys[105];

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;

    for (int i = 0; i < m; ++i) {
        int u, v; ll w; cin >> u >> v >> w;
        add(u, v, w);
    }

    for (int i = 0; i < k; ++i) cin >> keys[i];
    if (k == 1) { cout << -1 << "\n"; return 0; }

    for (int i = 0; i < k; ++i) {
        for (int j = 1; j <= n; ++j) dist[j] = INF;
        priority_queue<Node> pq;

        int start = keys[i];
        dist[start] = 0;
        pq.push({0, start});

        while (!pq.empty()) {
```

```
            Node cur = pq.top(); pq.pop();
            int u = cur.u; ll d = cur.d;
            if (d > dist[u]) continue;

            for (int j = head[u]; j; j = e[j].nxt) {
                int v = e[j].to; ll w = e[j].w;
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    pq.push({dist[v], v});
                }
            }
        }

        ll mn = INF;
        for (int j = 0; j < k; ++j) {
            if (i == j) continue;
            mn = min(mn, dist[keys[j]]);
        }
        cout << mn << "\n";
    }
    return 0;
}
```

测试点13~20：$n \le 100000, m \le 200000, k \le n$

思路

这里正解部分，k达到了10^5，不能直接对每个关键点跑 Dijkstra，否则复杂度会爆炸，我们需要使用二进制分组结合反向图多源 Dijkstra 的技巧。

1. **反向图的性质**：在原图中从u到v的最短路，等价于在反向图中从v到u的最短路。因此，在反向图中以集合S为起点跑多源 Dijkstra，求出的$dist[i]$就是原图中从i出发，到S中任意点的最短路。
2. **二进制分组思想**：对于任意两个不同的关键点，其对应的下标（编号）在二进制下至少有一位不同，我们可以按二进制每一位$b \in [0, \log_2 k]$，在第b轮中，将关键点按照第b位是否为 1 分为 group0 和 group1。
3. **多源Dijkstra执行**：
   - 在反向图上，以 group0 中的所有点为源点（距离为0）跑一次多源 Dijkstra，此时对于 group1 中任意的点x，$dist[x]$就等于x出发至少能到一位 group0 中的最短距离。
   - 对称地，在反向图上再以 group1 为源点跑多源 Dijkstra，更新 group0 中各关键点到的答案。
4. **正确性**：任意两点p_i, p_j，必然存在至少一位二进制位不同，在那一轮分组中，它们必然被分到不同的组，它们之间的最短路必然被计算到。总轮数为$\log_2 k$，每轮跑 2 次多源 Dijkstra，总时间复杂度为$O(m \log n \log k)$。

复杂度

时间复杂度$O(m \log n \log k)$，空间复杂度$O(n + m)$。

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
typedef long long ll;
const ll INF = 4e18;
const int MAXN = 100005;
const int MAXM = 200005;
// 1. 链式前向星（全局数组建反向图）
int head[MAXN], cnt = 0;
struct Edge {
    int to, nxt;
    ll w;
} e[MAXM];
void add(int u, int v, ll w) {
    e[++cnt] = {v, head[u], w};
    head[u] = cnt;
}
struct Node {
    ll d;
    int u;
    bool operator<(const Node& o) const {
        return d > o.d;
    }
};
int n, m, k;
int keys[MAXN];
ll dist[MAXN], ans[MAXN];
void dijkstra(int b, int val) {
    for(int i = 1; i <= n; ++i) dist[i] = INF;
    priority_queue<Node> pq;
    for(int i = 0; i < k; ++i) {
        if(((i >> b) & 1) == val) {
            int u = keys[i];
            dist[u] = 0;
            pq.push({0, u});
        }
    }
    while(!pq.empty()) {
        Node cur = pq.top();
        pq.pop();
        int u = cur.u;
        ll d = cur.d;
        if(d > dist[u]) continue;
        for(int i = head[u]; i; i = e[i].nxt) {
            int v = e[i].to;
            ll w = e[i].w;
            if(d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    for(int i = 0; i < k; ++i) {
        if(((i >> b) & 1) != val) {
            if(dist[keys[i]] < ans[i])
                ans[i] = dist[keys[i]];
        }
    }
```

```cpp
}
int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> m >> k;
    for(int i = 0; i < m; ++i) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        add(v, u, w); // 反向图建边：原图 u->v 变为反向图 v->u
    }
    for(int i = 0; i < k; ++i) cin >> keys[i];
    if(k == 1) {
        cout << -1 << "\n";
        return 0;
    }
    for(int i = 0; i < k; ++i) ans[i] = INF;
    // 计算所需二进制位数
    int L = 0;
    while((1 << L) < k) L++;
    // 二进制分组枚举
    for(int b = 0; b < L; ++b) {
        dijkstra(b, 0); // 以第 b 位为 0 的点作为源点，更新第 b 位为 1 的点
        dijkstra(b, 1); // 以第 b 位为 1 的点作为源点，更新第 b 位为 0 的点
    }
    for(int i = 0; i < k; ++i) {
        cout << ans[i] << "\n";
    }
    return 0;
}
```

另解：反图+分层图最短路

思路

上一份做法通过二进制分组，跑 $2\log k$ 次多源 Dijkstra，利用"任意两个关键点必然在某一位上分属不同组"的性质覆盖所有关键点对。但如果换一个角度，我们不需要真的把每一对关键点分开算，只需要保证**每个关键点都能找到离它最近的那个"别人"**即可。

题目的要求是：对每个关键点 p，求它到**其他任意一个关键点**的最短距离的最小值。注意这里"其他"二字是关键——也就是我们要找的是每个关键点的**第二近的关键点**（第一近的显然是它自己，距离为 0）。

于是问题转化为：**对每个关键点，求它到最近的其他关键点的距离。**这可以在一次多源 Dijkstra 中解决，只要每个点维护**两个最近的、来源不同的源点**即可。

1. **反向图的性质**：在反向图上以所有关键点为源点跑多源 Dijkstra，求出每个点的最短距离 $dist[i]$，这个距离等于原图中从 i 出发到最近关键点的距离。
2. **保留两个不同来源**：对于关键点 p 自身而言，最近的源点一定是它自己，距离为 0；我们真正想要的是它**第二近的来源。**因此在多源 Dijkstra 中，每个点维护：
   - $d_1[i], s_1[i]$：最近来源的距离和编号；
   - $d_2[i], s_2[i]$：第二近**且来源不同于** $s_1[i]$ 的距离和编号。
3. **更新的规则**：当从源点 s 松弛点 v 时，
   - 如果 $s = s_1[v]$，只需尝试用 nd 更新 $d_1[v]$；
   - 如果 $s = s_2[v]$，只需尝试用 nd 更新 $d_2[v]$；

- 如果 s 是一个新的来源：
  - 若 $nd < d_1[v]$，则原来的 (d_1, s_1) 下移成为 (d_2, s_2)，新的成为 (d_1, s_1)；
  - 否则若 $nd < d_2[v]$，直接替换 (d_2, s_2)。

4. **答案**：对于关键点 p，$d_2[p]$ 就是它到最近的其他关键点的距离。
5. **正确性**：多源 Dijkstra 的性质保证：当算法结束时，$d_1[u]$ 是 u 到最近关键点的距离，$d_2[u]$ 是 u 到次近的、来源不同的关键点的距离。对于关键点 p 而言，它自身的来源距离为 0，因此 $d_2[p]$ 必然来自另一个关键点 q，且是所有其他关键点里离 p 最近的那个。

这种写法的好处是：**只需要一次多源 Dijkstra**，时间复杂度 $O(m \log n)$，不依赖 k 的大小；而且逻辑比二进制分组更加直观。代价是需要维护每个点的两个来源，常数略大，但复杂度更优。

复杂度

时间复杂度 $O(m \log n)$，空间复杂度 $O(n + m)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = (ll)4e18;
const int MAXN = 100005, MAXM = 200005;
int head[MAXN], to[MAXM], nxt[MAXM], ecnt;
ll wt[MAXM];
int n, m, k, key_[MAXN];
ll d1[MAXN], d2[MAXN];
int s1_[MAXN], s2_[MAXN];
void add(int v, int u, ll w) {                  // 反图
    to[++ecnt] = u; wt[ecnt] = w; nxt[ecnt] = head[v]; head[v] = ecnt;
}
struct St { ll d; int s, u; };
struct Cmp { bool operator()(const St &a, const St &b) const { return a.d > b.d; } };
int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    cin >> n >> m >> k;
    for (int i = 0; i < m; ++i) { int u, v; ll w; cin >> u >> v >> w; add(v, u, w); }
    for (int i = 0; i < k; ++i) cin >> key_[i];
    if (k == 1) { cout << -1 << '\n'; return 0; }
    for (int i = 1; i <= n; ++i) { d1[i] = d2[i] = INF; s1_[i] = s2_[i] = 0; }
    priority_queue<St, vector<St>, Cmp> pq;
    for (int i = 0; i < k; ++i) {
        int p = key_[i];
        d1[p] = 0; s1_[p] = p;
        pq.push((St){0, p, p});
    }
    while (!pq.empty()) {
        St c = pq.top(); pq.pop();
        ll d = c.d; int s = c.s, u = c.u;
        if (!((s1_[u] == s && d1[u] == d) || (s2_[u] == s && d2[u] == d))) continue;
        for (int e = head[u]; e; e = nxt[e]) {
            int v = to[e]; ll nd = d + wt[e];
            if (s1_[v] == s) {
```

```
                if (nd < d1[v]) { d1[v] = nd; pq.push((St){nd, s, v}); }
            } else if (s2_[v] == s) {
                if (nd < d2[v]) {
                    d2[v] = nd;
                    pq.push((St){nd, s, v});
                    if (d2[v] < d1[v]) {            // 维持 d1 <= d2
                        ll td = d1[v]; d1[v] = d2[v]; d2[v] = td;
                        int ts = s1_[v]; s1_[v] = s2_[v]; s2_[v] = ts;
                        if (d2[v] < INF) pq.push((St){d2[v], s2_[v], v});
                    }
                }
            } else if (nd < d1[v]) {                // 新来源顶掉第一近
                d2[v] = d1[v]; s2_[v] = s1_[v];      // 旧的降级成第二近
                d1[v] = nd; s1_[v] = s;
                pq.push((St){nd, s, v});
                if (d2[v] < INF) pq.push((St){d2[v], s2_[v], v});   // 降级的也要入队
            } else if (nd < d2[v]) {
                d2[v] = nd; s2_[v] = s;
                pq.push((St){nd, s, v});
            }
        }
    }
    for (int i = 0; i < k; ++i) cout << (d2[key_[i]] >= INF / 2 ? -1 : d2[key_[i]]) << '\n';
    return 0;
}
```

咒文（charm）

本题正解所考察的知识点主要包括 Manacher、扫描线、线段树区间加等差数列、树状数组、前缀和与差分。其中“Manacher”属于“字符串算法”，NOI 大纲标注学习难度系数为 7；“扫描线”属于“算法策略”，难度系数为 7；“线段树”“树状数组”属于“数据结构”，难度系数为 6；“前缀和”难度系数为 3，“差分”难度系数为 4。本题所涉知识点均在 NOI 大纲 CSP-S 提高级范围内。

测试点 1~2：$n, q \leq 200$

思路

数据范围很小，直接暴力。对每个询问，枚举区间内每个中心 i，向两边扩展，统计可用半径 K_i，累加。

复杂度

单次询问 $O(n^2)$，总 $O(qn^2)$，空间 $O(n)$。

```
while(q--){
    int L, R;
    cin >> L >> R;
    long long ans = 0;
    for (int i = L; i <= R; ++i) {
        int k = 0;
        while (i - k >= L && i + k <= R && S[i - k] == S[i + k])
++k,ans+=k*2-1;
    }
    cout << ans % (1LL << 32) << '\n';
}
```

测试点 3~5：$n, q \le 3000$

思路

先跑一遍 Manacher 求出每个中心的 $p[i]$。

此时中心 i 实际可用的半径受到三个限制：

1. 本身最大半径是 $p[i]$；
2. 向左不能超过 L，最多能扩展 $i - L + 1$ 个字符；
3. 向右不能超过 R，最多能扩展 $R - i + 1$ 个字符。

对每个询问，枚举中心 $i \in [L, R]$，直接计算：$K_i = \min(p[i],\ i - L + 1,\ R - i + 1)$

那么以 i 为中心的奇回文半径可以是 $1, 2, \ldots, k$。
半径为 1 时长度为 1，半径为 2 时长度为 3，半径为 3 时长度为 5，……，半径为 k 时长度为 $2k - 1$。

这些长度全部加起来是：$1 + 3 + 5 + \cdots + (2k - 1) = k^2$

所以中心 i 对答案的贡献就是 k^2。
把所有中心 $i \in [L, R]$ 的贡献累加即可。

复杂度

Manacher 为 $O(n)$，每次询问为 $O(n)$，总 $O(n + qn)$，空间 $O(n)$。

```
// Manacher 求 p[1..n]
vector<int> p(n + 1);
int l = 1, r = 1;
for (int i = 1; i <= n; ++i) {
    p[i] = 1;
    if (i <= r) p[i] = min(p[l + r - i], r - i + 1);
    while (i - p[i] >= 1 && i + p[i] <= n && S[i - p[i]] == S[i + p[i]])
++p[i];
    if (i + p[i] - 1 > r) {
        l = i - p[i] + 1;
        r = i + p[i] - 1;
    }
}
```

```
13
14  for (int id = 0; id < q; ++id) {
15      int L, R;
16      cin >> L >> R;
17      long long ans = 0;
18      for (int i = L; i <= R; ++i) {
19          int k = min({p[i], i - L + 1, R - i + 1});
20          ans += 1LL * k * k;
21      }
22      cout << ans % (1LL << 32) << '\n';
23  }
```

测试点 6~8：$n, q \leq 3 \times 10^5$，字符串 S 仅包含一种字符

思路

当字符串所有字符都相同时，任意一个连续子串都是回文。
题目只统计奇回文，因此只需要统计区间内所有奇数长度子串的长度之和。

设询问区间长度为 $m = R - L + 1$。
对于长度为 $d = 2k - 1$ 的奇数长度子串，它在区间内出现的次数是
$m - d + 1 = m - (2k - 1) + 1 = m - 2k + 2$。
所以总贡献为 $S(m) = \sum_{k=1}^{\lceil m/2 \rceil} (2k - 1)(m - 2k + 2)$。

令 $K = \lceil \frac{m}{2} \rceil$，利用求和公式 $\sum_{k=1}^{K} k = \frac{K(K+1)}{2}$ 和 $\sum_{k=1}^{K} k^2 = \frac{K(K+1)(2K+1)}{6}$，可以把 $S(m)$ 化简为 $S(m) = 2(m+3)\sum_{k=1}^{K} k - 4\sum_{k=1}^{K} k^2 - K(m+2)$。

这样每个询问就可以在 $O(1)$ 时间内算出答案。

复杂度

每次询问 $O(1)$，总时间复杂度 $O(n+q)$，空间复杂度 $O(1)$。

测试点 9~10：$n, q \leq 3 \times 10^5$，所有询问满足 $L = 1$

思路

询问变为 $[1, R]$。此时 $i - L + 1 = i$，而 $p[i] \leq i$，故左端不限制半径：
$K_i = \min(p[i],\ R - i + 1)$

按 R 从大到小处理。
若 $rb[i] \geq R$，则 $p[i] \geq R - i + 1$，此时 $K_i = R - i + 1$，称中心 i 被“激活”。
否则 $K_i = p[i]$。

答案可写成：$\text{ans} = \sum_{i=1}^{R} p[i]^2 + \sum_{\text{激活}} \left((R - i + 1)^2 - p[i]^2 \right)$

其中 $\sum_{i=1}^{R} p[i]^2$ 用前缀和预处理。

对激活中心：$(R - i + 1)^2 = (R+1)^2 - 2(R+1)i + i^2$

用树状数组维护激活中心的个数 cnt、$\sum i$、$\sum i^2$、$\sum p[i]^2$。
查询前缀 $[1, R]$ 后：$\sum_{\text{激活}} (R - i + 1)^2 = cnt \cdot (R+1)^2 - 2(R+1)\sum i + \sum i^2$

最终：$\text{ans} = \text{preP2}[R] + cnt \cdot (R+1)^2 - 2(R+1)\sum i + \sum i^2 - \sum p[i]^2$

复杂度

$O((n+q)\log n)$，空间 $O(n)$。

测试点 11~12：$n, q \le 3 \times 10^5$，所有询问满足 $R = n$

思路

询问为 $[L, n]$，与 $L = 1$ 对称。
翻转字符串 S，原询问变为 $[1,\ n-L+1]$，直接套用测试点 9~10 的树状数组做法。

复杂度

时间复杂度 $O((n+q)\log n)$，空间 $O(n)$。

测试点 13~20：$n, q \le 3 \times 10^5$

思路

现在要处理一般的 `[L,R]`。回到最原始的那个式子：

$$K_i = \min\left(rad[i],\ i-L+1,\ R-i+1\right)$$

三个约束同时取最小，很难直接批量算。办法是**把每个奇回文看成平面上的一个点。**

一个奇回文 `[l,r]` 对应平面上的一个点 `(l, r)`。询问"完全包含在 `[L,R]` 内"就是条件 $L \le l \le r \le R$，这在 `(l,r)` 平面上是一个直角三角形。

再画一条直线 $l + r = L + R = D$，把三角形切成两块：

- **区域 A**：$l + r \le D$。这一块里，由 $l \ge L$ 立刻得到 $r = (l+r) - l \le D - L = R$，也就是说 `r ≤ R` 自动成立，只需要检查 $l \ge L$。
- **区域 B**：$l + r \ge D + 1$。对称地，由 $r \le R$ 得到 $l = (l+r) - r \ge D + 1 - R = L + 1 > L$，也就是说 `l ≥ L` 自动成立，只需要检查 $r \le R$。

因为奇回文的 $l + r = 2i$ 一定是偶数，$l + r \le D$ 和 $l + r \ge D + 1$ 之间不会漏掉也不会重复任何点，两块正好不重不漏。

于是答案拆成两部分，各自只需要一个单侧条件：

$$\text{ans} = \underbrace{\sum_{i\in\text{区域 A}} \text{长度}_i}_{\text{查 } l_i \ge L \text{ 的后缀和}} + \underbrace{\sum_{i\in\text{区域 B}} \text{长度}_i}_{\text{查 } r_i \le R \text{ 的前缀和}}$$

下面看怎么用扫描线维护这两个和。

区域 A：按 D 从小到大处理询问。处理 D 时，把所有满足 $2i \le D$ 的中心 `i` 加进来。中心 `i` 贡献的回文，其左端点 `l` 落在 `[lb[i], i]`，长度 $= 2(i-l)+1 = -2l + (2i+1)$。所以这是一次**关于 `l` 的区间加等差数列**（公差 -2），作用区间 `[lb[i], i]`。加完之后询问只要取 `l ≥ L` 的区间和。

区域 B：按 D 从大到小处理询问。阈值是 $D+1$，加进所有满足 $2i \ge D+1$ 的中心 `i`。中心 `i` 的回文右端点 `r` 落在 `[i, rb[i]]`，长度 $= 2(r-i)+1 = 2r+(1-2i)$，也是一次区间加等差数列（公差 $+2$），作用区间 `[i, rb[i]]`。询问取 `r ≤ R` 的区间和。

两次扫描各做 n 次区间加、q 次区间求和，用**线段树维护"区间加 $kx+b$ + 区间求和"**：每个结点记 `sum`（区间和）、`posSum`（区间内下标之和，建树时算一次，之后不变）、以及懒标记 `(k, b)` 表示这一段每个位置都要加 $kx+b$。给整段打标记时

$$sum \mathrel{+}= k \cdot posSum + b \cdot \text{区间长度}$$

下传时把 `(k, b)` 原样传给两个儿子即可。

复杂度

- 时间复杂度：$O((n+q)\log n)$。
- 空间复杂度：$O(n)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 300005;
const ll mod = 1LL<<32;
int n, q, rad[MAXN], lb[MAXN], rb[MAXN], qL[MAXN], qR[MAXN], ord[MAXN];
ll A_[MAXN], B_[MAXN], PSq[MAXN];
ll sum_[4 * MAXN], posSum[4 * MAXN], lzK[4 * MAXN], lzB[4 * MAXN];
string s;
void build(int p, int l, int r) {
    posSum[p] = 1LL * (l + r) * (r - l + 1) / 2;
    sum_[p] = lzK[p] = lzB[p] = 0;
    if (l == r) return;
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
}
void apply(int p, int l, int r, ll k, ll b) {
    sum_[p] += k * posSum[p] + b * (r - l + 1);
    lzK[p] += k; lzB[p] += b;
}
void push(int p, int l, int r) {
    if (!lzK[p] && !lzB[p]) return;
    int mid = (l + r) >> 1;
    apply(p << 1, l, mid, lzK[p], lzB[p]);
    apply(p << 1 | 1, mid + 1, r, lzK[p], lzB[p]);
    lzK[p] = lzB[p] = 0;
}
void add(int p, int l, int r, int ql, int qr, ll k, ll b) {
    if (ql <= l && r <= qr) { apply(p, l, r, k, b); return; }
    push(p, l, r);
    int mid = (l + r) >> 1;
    if (ql <= mid) add(p << 1, l, mid, ql, qr, k, b);
    if (qr > mid) add(p << 1 | 1, mid + 1, r, ql, qr, k, b);
    sum_[p] = sum_[p << 1] + sum_[p << 1 | 1];
}
ll query(int p, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return 0;
```

```cpp
    if (ql <= l && r <= qr) return sum_[p];
    push(p, l, r);
    int mid = (l + r) >> 1;
    return query(p << 1, l, mid, ql, qr) + query(p << 1 | 1, mid + 1, r, ql, qr);
}
int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    cin >> n >> q >> s; s = " " + s;
    for (int i = 1, l = 1, r = 0; i <= n; ++i) {
        int k = (i > r) ? 1 : min(rad[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) ++k;
        rad[i] = k;
        if (i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }
        lb[i] = i - k + 1; rb[i] = i + k - 1;
    }
    for (int i = 0; i < q; ++i) cin >> qL[i] >> qR[i];
    for (int i = 0; i < q; ++i) ord[i] = i;
    // 区域 A: D 升序, 插入 2i<=D 的中心, 对 l∈[lb[i],i] 加 -2l+(2i+1), 查 l>=L
    sort(ord, ord + q, [](int x, int y) { return qL[x] + qR[x] < qL[y] + qR[y]; });
    build(1, 1, n);
    for (int ptr = 1, p = 0; p < q; ++p) {
        int id = ord[p], D = qL[id] + qR[id];
        while (ptr <= n && 2 * ptr <= D) { add(1, 1, n, lb[ptr], ptr, -2, 2LL * ptr + 1); ++ptr; }
        A_[id] = query(1, 1, n, qL[id], n);
    }
    // 区域 B: D 降序, 插入 2i>=D+1 的中心, 对 r∈[i,rb[i]] 加 2r+(1-2i), 查 r<=R
    sort(ord, ord + q, [](int x, int y) { return qL[x] + qR[x] > qL[y] + qR[y]; });
    build(1, 1, n);
    for (int ptr = n, p = 0; p < q; ++p) {
        int id = ord[p], D = qL[id] + qR[id];
        while (ptr >= 1 && 2 * ptr >= D + 1) { add(1, 1, n, ptr, rb[ptr], 2, 1LL - 2 * ptr); --ptr; }
        B_[id] = query(1, 1, n, 1, qR[id]);
    }
    for (int i = 0; i < q; ++i) cout << (A_[i] + B_[i])% mod << '\n';
    return 0;
}
```

另解：半划分 + 激活 + 树状数组

上面的切法是按 $l + r$ 斜着切。也可以竖着切一刀：把询问区间按中点 $mid = \lfloor (L + R)/2 \rfloor$ 分成左右两半。

- 左半 $i \in [L, mid]$：因为 $i \le mid$，有 $i - L + 1 \le R - i + 1$，三个约束里右边那头永远卡不住，于是 $K_i = \min(rad[i],\ i - L + 1)$；
- 右半 $i \in (mid, R]$：对称地 $i - L + 1 \ge R - i + 1$，左边卡不住，于是 $K_i = \min(rad[i],\ R - i + 1)$。

每一半都只剩**单侧裁剪**，于是可以套“激活 + 树状数组”的套路：

- 左半按 `L` 升序扫描，当 $lb[i] \le L$ 时激活中心 `i`（这时 $rad[i] \ge i-L+1$，K_i 取后者）；
- 右半按 `R` 降序扫描，当 $rb[i] \ge R$ 时激活中心 `i`（这时 $K_i = R-i+1$）。

激活的中心单独算，没激活的直接用预处理的 $\sum rad^2$ 前缀和。

写的时候注意左半的展开：令 $u = i+1$，那么 $K_i = \min(rad[i], u-L)$，激活时 $K_i^2 = (u-L)^2 = u^2 - 2Lu + L^2$。所以树状数组维护激活集合的 $\sum(u^2 - rad^2)$、$\sum u$、个数，查询时

$$\text{左半} = \sum_{i=L}^{mid} rad[i]^2 + \left[\Sigma(u^2 - rad^2)\right] - 2L \cdot \Sigma u + L^2 \cdot cnt$$

右半同理，激活时 $K_i^2 = (R+1-i)^2 = (R+1)^2 - 2(R+1)i + i^2$，维护 $\sum rad^2$、$\sum i$、$\sum i^2$、个数。

复杂度

- 时间复杂度：$O((n+q)\log n)$，时间复杂度一样，但常数比线段树版小。
- 空间复杂度：$O(n)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
struct BIT {
    int n; vector<ll> t;
    void init(int n_) { n = n_; t.assign(n + 2, 0); }
    void add(int i, ll v) { for (; i <= n; i += i & -i) t[i] += v; }
    ll sum(int i) { ll r = 0; if (i > n) i = n; for (; i > 0; i -= i & -i) r += t[i]; return r; }
    ll range(int l, int r) { if (l > r) return 0; return sum(r) - sum(l - 1); }
};
int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, q; string s;
    cin >> n >> q >> s;
    s = " " + s;
    vector<int> rad(n + 2, 0), lb(n + 2, 0), rb(n + 2, 0);
    for (int i = 1, l = 1, r = 0; i <= n; ++i) {
        int k = (i > r) ? 1 : min(rad[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) ++k;
        rad[i] = k;
        if (i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }
        lb[i] = i - k + 1; rb[i] = i + k - 1;
    }
    vector<ll> PSq(n + 1, 0);                    // p[i]^2 前缀和
    for (int i = 1; i <= n; ++i) PSq[i] = PSq[i - 1] + 1LL * rad[i] * rad[i];
    vector<int> qL(q), qR(q), id(q);
    vector<ll> ans(q, 0);
    for (int i = 0; i < q; ++i) cin >> qL[i] >> qR[i];
    iota(id.begin(), id.end(), 0);
    // ---------- 左半：i ∈ [L, mid]，K_i = min(p[i], i-L+1) ----------
    // 激活条件 lb[i] <= L（L 从小到大扫，L 越大激活越多）
    vector<vector<int>> byLb(n + 1);
    for (int i = 1; i <= n; ++i) byLb[lb[i]].push_back(i);
    sort(id.begin(), id.end(), [&](int x, int y) { return qL[x] < qL[y]; });
```

```
    BIT b1, b2, b3;                                 // Σ(p²-(i+1)²), Σ(i+1), 个数
    b1.init(n); b2.init(n); b3.init(n);
    for (int curL = 1, p = 0; p < q; ++p) {
        int L = qL[id[p]];
        while (curL <= L) {                         // 激活所有 lb[i] == curL
            for (int i : byLb[curL]) {
                b1.add(i, 1LL * rad[i] * rad[i] - 1LL * (i + 1) * (i + 1));
                b2.add(i, i + 1);
                b3.add(i, 1);
            }
            ++curL;
        }
        int R = qR[id[p]], mid = (L + R) / 2;
        ll tot = PSq[mid] - PSq[L - 1];             // Σ p[i]^2
        ll s1 = b1.range(L, mid), s2 = b2.range(L, mid), s3 = b3.range(L, mid);
        ans[id[p]] += tot - s1 - 2LL * L * s2 + 1LL * L * L * s3;
    }
    // ---------- 右半: i ∈ (mid, R], K_i = min(p[i], R-i+1) ----------
    // 激活条件 rb[i] >= R（R 从大到小扫，R 减小时激活增多）
    vector<vector<int>> byRb(n + 1);
    for (int i = 1; i <= n; ++i) byRb[rb[i]].push_back(i);
    sort(id.begin(), id.end(), [&](int x, int y) { return qR[x] > qR[y]; });
    BIT c1, c2, c3, c4;                             // Σ p², Σ i, Σ i², 个数
    c1.init(n); c2.init(n); c3.init(n); c4.init(n);
    for (int curR = n, p = 0; p < q; ++p) {
        int R = qR[id[p]];
        while (curR >= R) {                         // 激活所有 rb[i] == curR
            for (int i : byRb[curR]) {
                c1.add(i, 1LL * rad[i] * rad[i]);
                c2.add(i, i);
                c3.add(i, 1LL * i * i);
                c4.add(i, 1);
            }
            --curR;
        }
        int L = qL[id[p]], mid = (L + R) / 2;
        ll tot = PSq[R] - PSq[mid];                 // Σ p[i]^2
        ll s1 = c1.range(mid + 1, R), s2 = c2.range(mid + 1, R);
        ll s3 = c3.range(mid + 1, R), s4 = c4.range(mid + 1, R);
        // (R-i+1)^2 = (R+1)^2 - 2(R+1)i + i^2
        ll active = 1LL * (R + 1) * (R + 1) * s4 - 2LL * (R + 1) * s2 + s3;
        ans[id[p]] += (tot - s1) + active;
    }
    for (int i = 0; i < q; ++i) cout << ans[i] << '\n';
    return 0;
}
```

完全平方数（square）

本题正解所考察的知识点主要包括模拟、递推、前缀和、整数平方根与分块缓存优化。其中"模拟"属于"入门算法"，NOI大纲标注学习难度系数为1；"递推"属于"基础算法"，大纲标注学习难度系数为3；"前缀和"属于"算法策略"，大纲标注学习难度系数为3；"整数平方根"涉及"数学库常用函数"，大纲标注学习难度系数为1；"分块缓存优化"属于"算法策略"与"数据结构"，大纲标注学习难度系数为6。本题所涉知识点的难度在NOI大纲所规定的CSP-S（提高级）难度系数范围内。

测试点 1：$T = 1$，$L = R \le 1000$

思路

只有一个询问且 $L = R$，只有一个数，按定义一步步减就行。每次减去 $\lfloor \sqrt{x} \rfloor^2$，减的次数就是答案。

注意 $\lfloor \sqrt{x} \rfloor$ 要用整数平方根，直接 `(long long)sqrt(x)` 在 `x` 很大时会有浮点误差，稳妥写法是先取 `sqrt` 再上下修正一两步。

复杂度

- 时间复杂度：$O(f(x))$，约 $O(\sqrt[3]{x})$ 级别，很小。
- 空间复杂度：$O(1)$。

```cpp
ll x = L, cnt = 0;
while (x > 0) {
    ll s = sqrt(x);
    while ((s + 1) * (s + 1) <= x) ++s;
    while (s * s > x) --s;
    x -= s * s;
    ++cnt;
}
cout << cnt << '\n';
```

测试点 2：$T \le 10$，$R - L \le 1000$，$R \le 10^6$

思路

区间长度很小，直接枚举区间 $[L, R]$ 内的每个整数 x，对每个 x 模拟过程，累加步数。

复杂度

单组询问 $O((R - L) \log R)$，总时间复杂度 $O(T \times (R - L) \log R)$，空间复杂度 $O(1)$。

```cpp
ll ans = 0;
for (ll x = L; x <= R; ++x) {
    ll v = x;
    while (v > 0) {
        ll s = sqrt(v);
        while ((s + 1) * (s + 1) <= v) ++s;
        while (s * s > v) --s;
        v -= s * s;
        ++ans;
    }
}
cout << ans << '\n';
```

测试点 3：$T \le 1000$，$R \le 10^6$

思路

R 只有 10^6，可以先把 10^6 以内 $f(i)$（表示数字 i 的步数）预处理出来，以及预处理前缀和 $S(i) = \sum_{x=1}^{i} f(x)$。

递推式：$\boxed{f(i) = 1 + f(i - \lfloor \sqrt{i} \rfloor^2)}$。对于每个询问，答案为 $S(R) - S(L-1)$。

复杂度

预处理 $O(R_{\max})$，每次询问 $O(1)$，总时间复杂度 $O(10^6 + T)$，空间复杂度 $O(10^6)$。

```cpp
const int MAXN = 1000000;
vector<int> f(MAXN + 1), S(MAXN + 1);
for (int i = 1; i <= MAXN; ++i) {
    int s = sqrt(i);
    while ((s + 1) * (s + 1) <= i) ++s;
    while (s * s > i) --s;
    f[i] = 1 + f[i - s * s];
    S[i] = S[i - 1] + f[i];
}
cout << S[R] - S[L - 1] << '\n';
```

测试点 4：$T \le 10^5$，$R \le 10^9$

思路

无法预处理到 10^9，但可以化简 $S(n)$ 递推式：

- 对于任意整数 n，令 $s = \lfloor \sqrt{n} \rfloor$。这里 s 表示不超过 $\sqrt{n}$ 的最大整数，也就是 n 的整数平方根。那么有 $s^2 \le n < (s+1)^2$。
- 将 1 到 n 分成两段：第一段是 1 到 $s^2 - 1$，第二段是 s^2 到 n。

- 对于第二段中的任意整数 x，因为 s^2 是不超过 x 的最大完全平方数，所以第一步一定会减去 s^2，剩下的数是 $x-s^2$。这个剩余值满足 $0 \le x-s^2 \le n-s^2$。又因为 $n < (s+1)^2$，所以 $n-s^2 < (s+1)^2-s^2 = 2s+1$，即 $n-s^2 \le 2s$。
- 因此，对于第二段中的每个 x，有 $f(x) = 1 + f(x-s^2)$。于是第二段的总步数之和为：
- $\sum_{x=s^2}^{n} f(x) = \sum_{r=0}^{n-s^2}(1+f(r)) = (n-s^2+1) + S(n-s^2)$

所以得到 $S(n)$ 的递推式：$\boxed{S(n) = S(s^2-1) + (n-s^2+1) + S(n-s^2)}$

定义利用 $T(s) = S(s^2-1)$，可以写成：$S(n) = T(s) + (n-s^2+1) + S(n-s^2)$，其中 $s = \lfloor \sqrt{n} \rfloor$，且 $n-s^2 \le 2s$。

接下来推导 $T(m)$ 的递推式：

- 考虑 $T(m) = S(m^2-1)$。令 $n = m^2-1$，则 $s = \lfloor \sqrt{n} \rfloor = m-1$。代入式子：
- $S(m^2-1) = S((m-1)^2-1) + ((m^2-1)-(m-1)^2+1) + S((m^2-1)-(m-1)^2)$
- 计算中间项：$(m^2-1)-(m-1)^2+1 = 2m-1$。
- 计算最后一项：$(m^2-1)-(m-1)^2 = 2m-2$。

所以：$\boxed{T(m) = T(m-1) + (2m-1) + S(2m-2)}$

注意 $T(0) = S(-1) = 0$，$T(1) = S(0) = 0$，递推从 $m=2$ 开始。

$$\boxed{S(n) = T(s) + (n-s^2+1) + S(n-s^2)}, \qquad s = \lfloor \sqrt{n} \rfloor$$

这个式子是整个题的发动机，而且它有个特别好用的性质：

$$n - s^2 < (s+1)^2 - s^2 = 2s \quad \Rightarrow \quad n - s^2 \le 2s$$

也就是说，递归一层之后，自变量从 n 一下掉到不超过 $2s \approx 2\sqrt{n}$ 的规模。$\sqrt{n}$ 相对 n 小太多了，所以只要预处理出小规模的表，任意大的 n 都能一两步查到表上。后面几档的分界线，本质上就是看“$2s$ 有多大、表要开多大”。

看数量级：

- $s = \lfloor \sqrt{R} \rfloor \le \lfloor \sqrt{10^9} \rfloor = 31623$;
- $n - s^2 \le 2s \le 63246$。

所以只要预处理 S 到 2×10^6、T 到 10^6，式子里的 $T(s)$（$s \le 31623 < 10^6$）和 $S(n-s^2)$（$\le 63246 < 2 \times 10^6$）就都在表里。每次询问 $O(1)$。

预处理 T 用递推 $T(m) = T(m-1) + (2m-1) + S(2m-2)$，注意 $2m-2 \le 2 \times 10^6$，也在 s 的表内，正好接得上。

复杂度

预处理 $O(2 \times 10^6)$，每次询问 $O(1)$，总时间复杂度 $O(2 \times 10^6 + T)$，空间复杂度 $O(2 \times 10^6)$。

测试点 5~6：$T \le 10^5$，$R \le 10^{12}$

思路

式子完全没变，只是数字变大。重新核一下两个上界：

- $s = \lfloor \sqrt{R} \rfloor \le 10^6$；
- $n - s^2 \le 2s \le 2 \times 10^6$。

跟上一段是同一份代码，只要全程用 `long long` 就行（`R` 到 10^{12}，s^2 也不过 10^{12}，不会溢出）。

复杂度

预处理 $O(2 \times 10^6)$，每次询问 $O(1)$，总时间复杂度 $O(2 \times 10^6 + T)$，空间复杂度 $O(2 \times 10^6)$。

```cpp
const int MAXN = 2000000;
const int MAXM = 1000000;
vector<ll> S(MAXN + 1), T(MAXM + 1);
vector<int> f(MAXN + 1);
// 预处理 f 和 S
for (int i = 1; i <= MAXN; ++i) {
    int s = sqrt(i);
    while ((s + 1) * (s + 1) <= i) ++s;
    while (s * s > i) --s;
    f[i] = 1 + f[i - s * s];
    S[i] = S[i - 1] + f[i];
}
// 预处理 T
T[0] = 0; T[1] = 0;
for (int m = 2; m <= MAXM; ++m) {
    T[m] = T[m - 1] + (2LL * m - 1) + S[2 * m - 2];
}
// 查询
ll solve(ll n) {
    if (n <= 0) return 0;
    if (n <= MAXN) return S[n];
    ll s = isqrt(n);
    return T[s] + (n - s * s + 1) + S[n - s * s];
}
cout << solve(R) - solve(L - 1) << '\n';
```

测试点 7：$T \le 10^5$，$R \le 10^{16}$

思路

此时 $s = \lfloor \sqrt{R} \rfloor \le 10^8$，无法预处理 T 到 10^8（内存超限）。

但注意一个关键事实：算 $T(k)$ 只需要 $S(2k-2)$，而 $2k-2$ 的整数平方根只有

$$u = \lfloor \sqrt{2k-2} \rfloor \le \sqrt{2 \times 10^8} \approx 14142$$

小得离谱。所以可以这样：

1. 预处理 S 到 4×10^6、T 到 2×10^6（覆盖所有 $u \le 14142$ 的情形）；
2. 把所有询问里真正需要的、超过 2×10^6 的那些 `s` 收集起来排序去重；

3. 从 $2\times10^6+1$ 开始往上**逐个递推** $T(k)=T(k-1)+(2k-1)+S(2k-2)$，其中的 $S(2k-2)$ 用核心递推式现算（一层递归就能落到表上）；
4. 只把“需要的”那些 $T(k)$ 记进 `map`。

复杂度

预处理 $O(4\times10^6+2\times10^6)$，在线递推总量 $O(s_{\max}-2\times10^6)\le10^8$，每次询问递归深度 $O(1)$。总时间复杂度约 $O(10^8)$，空间复杂度约 50 MB。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int MAXN = 4000000;      // S 预处理到 4e6
const int MAXM = 2000000;      // T 预处理到 2e6
int f[MAXN + 1];
ll S[MAXN + 1], T[MAXM + 1];
ll isqrt(ll n) {
    if (n <= 0) return 0;
    ll x = sqrt((long double)n);
    while ((x + 1) * (x + 1) <= n) ++x;
    while (x * x > n) --x;
    return x;
}
unordered_map<ll, ll> TL;                                  // 在线递推出来的 T(k), k > MAXM
ll calc_S(ll n) {
    if (n <= 0) return 0;
    if (n <= MAXN) return S[n];
    ll s = isqrt(n);
    ll Ts = (s <= MAXM) ? T[s] : TL[s];
    return Ts + (n - s * s + 1) + calc_S(n - s * s);
}
int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    for (int i = 1; i <= MAXN; ++i) {
        int s = (int)isqrt(i);
        f[i] = 1 + f[i - s * s];
        S[i] = S[i - 1] + f[i];
    }
    for (int m = 2; m <= MAXM; ++m) T[m] = T[m - 1] + (2LL * m - 1) + S[2 * m - 2];
    int Q; cin >> Q;
    vector<pair<ll, ll>> qs(Q);
    vector<ll> need;
    for (int i = 0; i < Q; ++i) {
        ll L, R; cin >> L >> R; qs[i] = {L, R};
        ll a = isqrt(R), b = isqrt(L - 1);
        if (a > MAXM) need.push_back(a);
        if (b > MAXM) need.push_back(b);
    }
    sort(need.begin(), need.end());
    need.erase(unique(need.begin(), need.end()), need.end());
    ll curT = T[MAXM];
    size_t z = 0;
    for (ll k = MAXM + 1; z < need.size(); ++k) {
```

```
        curT += (2 * k - 1) + calc_S(2 * k - 2);
        while (z < need.size() && need[z] == k) { TL[k] = curT; ++z; }
    }
    for (int i = 0; i < Q; ++i)
        cout << calc_S(qs[i].second) - calc_S(qs[i].first - 1) << '\n';
    return 0;
}
```

测试点 8~10：$T \le 10^5$，$R \le 10^{18}$

思路

对于部分分7现在我们要处理 $R \le 10^{18}$ 的情况。此时最大的 $s = \lfloor \sqrt{R} \rfloor$ 可以达到 10^9。如果直接预处理 T 到 10^9，内存需要约 8 GB，不可行。

得想办法**一次算一整块**。

看 $T(k) = T(k-1) + (2k-1) + S(2k-2)$ 的递推式里那个 $S(2k-2)$：它的整数平方根 $u = \lfloor \sqrt{2k-2} \rfloor$ 在 `k` 变化一大段时是**不变的**。反过来说，u 相同的那些 `k` 会连成一段：

$$u^2 \le 2k-2 < (u+1)^2 \quad \Longleftrightarrow \quad \frac{u^2+2}{2} \le k < \frac{(u+1)^2+2}{2}$$

于是块内 u 固定，整块的增量可以拆成四部分一次算完：

所以 k 的左端点 $k_L = \left\lceil \frac{u^2+2}{2} \right\rceil$，右端点 $k_R = \left\lfloor \frac{(u+1)^2+1}{2} \right\rfloor$。块的长度约为 $u+1$，u 越大块越长。

在块内，我们要计算 $\boxed{\sum_{k=k_L}^{k_R} [T(k) - T(k-1)] = \sum_{k=k_L}^{k_R} [(2k-1) + S(2k-2)]}$

而 $S(2k-2)$ 可以展开为：

$$S(2k-2) = T(u) + (2k-1-u^2) + S(2k-2-u^2)$$

所以整个块的增量是：

$$\boxed{\Delta T = \underbrace{\sum_{k=k_L}^{k_R} (2k-1)}_{\text{等差数列}} + \underbrace{\sum_{k=k_L}^{k_R} T(u)}_{\text{常数}} + \underbrace{\sum_{k=k_L}^{k_R} (2k-1-u^2)}_{\text{等差数列减 } cnt\cdot u^2} + \underbrace{\sum_{k=k_L}^{k_R} S(2k-2-u^2)}_{\text{下标成等差数列的 } S \text{ 之和}}}$$

最后一项是关键：块内 $2k-2-u^2$ 是公差为 2 的等差数列，下标同奇偶。所以预处理两个**奇偶前缀和**：

$$pre_even[i] = S(0) + S(2) + \cdots + S(2i), \qquad pre_odd[i] = S(1) + S(3) + \cdots + S(2i+1)$$

就可以 $O(1)$ 查出任意一段同奇偶下标上的 S 之和。

预处理上限取 $\mathrm{LIM} = \lfloor \sqrt{2 \cdot s_{\max}} \rfloor + 10 \approx 44731$（$s_{max}$ 是所有询问里最大的那个 s），把 $S(n)$ 算到 $2 * LIM$、$T(m)$ 算到 LIM。这样递归 `calc_S` 时，由于 $n - s^2 \le 2s$，而 $s \le \max s$，所以 $n - s^2 \le 2 \max s$。递归一次后，新的 n' 的平方根 $\lfloor \sqrt{n'} \rfloor \le \sqrt{2 \max s} = \mathrm{LIM}$，所以第二次调用 calc_s 时一定在预处理范围内，直接查表即可。因此递归深度最多两层。

复杂度

- 预处理：$O(\text{LIMIT}) \approx O(4.5 \times 10^4)$
- 分块递推：块数 $O(\sqrt{\max s}) \approx O(4.5 \times 10^4)$
- 每次询问：递归深度 $O(\log \log n)$
- 总时间复杂度：$O(n^{\frac{1}{4}} + Q \log \log n)$，空间复杂度：$O(n^{\frac{1}{4}})$

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll isqrt(ll n) {
    if (n <= 0) return 0;
    ll x = sqrt((long double)n);
    while ((x + 1) * (x + 1) <= n) ++x;
    while (x * x > n) --x;
    return x;
}
int main() {
    ios::sync_with_stdio(0), cin.tie(0);
    int Q; cin >> Q;
    vector<pair<ll, ll>> qs(Q);
    ll max_s = 0;
    for (int i = 0; i < Q; ++i) {
        ll L, R; cin >> L >> R; qs[i] = {L, R};
        max_s = max(max_s, isqrt(R));
        max_s = max(max_s, isqrt(L - 1));
    }
    ll LIM = isqrt(2 * max_s) + 10;                // 预处理上限
    vector<ll> S(2 * LIM + 1, 0), T(LIM + 1, 0);
    vector<int> f(2 * LIM + 1, 0);
    for (int i = 1; i <= 2 * LIM; ++i) {
        int s = (int)isqrt(i);
        f[i] = 1 + f[i - s * s];
        S[i] = S[i - 1] + f[i];
    }
    for (int m = 2; m <= LIM; ++m) T[m] = T[m - 1] + (2LL * m - 1) + S[2 * m - 2];
    // 同奇偶下标上 S 的前缀和
    vector<ll> pre_even(LIM + 1, 0), pre_odd(LIM + 1, 0);
    pre_even[0] = S[0];
    for (int i = 1; i <= LIM; ++i) pre_even[i] = pre_even[i - 1] + S[2 * i];
    if (LIM >= 1) {
        pre_odd[0] = S[1];
        for (int i = 1; i < LIM; ++i) pre_odd[i] = pre_odd[i - 1] + S[2 * i + 1];
    }
    // 收集需要现算的 T 值
    vector<ll> need;
    for (int i = 0; i < Q; ++i) {
        ll a = isqrt(qs[i].second), b = isqrt(qs[i].first - 1);
        if (a > LIM) need.push_back(a);
        if (b > LIM) need.push_back(b);
    }
    sort(need.begin(), need.end());
    need.erase(unique(need.begin(), need.end()), need.end());
```

```cpp
    map<ll, ll> TL;
    ll cur_k = LIM + 1, cur_T = T[LIM];
    for (size_t z = 0; z < need.size(); ++z) {
        while (cur_k <= need[z]) {
            ll u = isqrt(2 * cur_k - 2);                    // 块内 u 固定
            ll kL = (u * u + 2) / 2; if (kL < cur_k) kL = cur_k;
            ll kR = (u * u + 2 * u + 2) / 2; if (kR > need[z]) kR = need[z];
            ll cnt = kR - kL + 1;
            ll sum2k = cnt * (kL + kR - 1);                 // Σ(2k-1)
            ll r0 = 2 * kL - 2 - u * u, sumS;
            if (r0 % 2 == 0) {
                ll s0 = r0 / 2;
                sumS = pre_even[s0 + cnt - 1] - (s0 > 0 ? pre_even[s0 - 1] :
0);
            } else {
                ll s0 = (r0 - 1) / 2;
                sumS = pre_odd[s0 + cnt - 1] - (s0 > 0 ? pre_odd[s0 - 1] :
0);
            }
            cur_T += 2 * sum2k + cnt * T[u] - cnt * u * u + sumS;
            cur_k = kR + 1;
        }
        TL[need[z]] = cur_T;
    }
    function<ll(ll)> calc_S = [&](ll n) -> ll {
        if (n <= 0) return 0;
        if (n <= 2 * LIM) return S[n];
        ll s = isqrt(n);
        ll Ts = (s <= LIM) ? T[s] : TL[s];
        return Ts + (n - s * s + 1) + calc_S(n - s * s);
    };
    for (int i = 0; i < Q; ++i)
        cout << calc_S(qs[i].second) - calc_S(qs[i].first - 1) << '\n';
    return 0;
}
```

浙江余博士教编程课程特色：

1、全系列课程：从语法**入门**到**省选**课程

2、3 大特色：浙大冠军博士团队特色、竞赛特色、融合中高考学科的课程特色

3、浙江信奥名师直播授课

余博士：杭州市高层次人才、浙大最高荣誉竺可桢奖学金、浙江大学校外合作导师、世界 500 强 15 年研发和管理经验、9 年编程任教经验。

